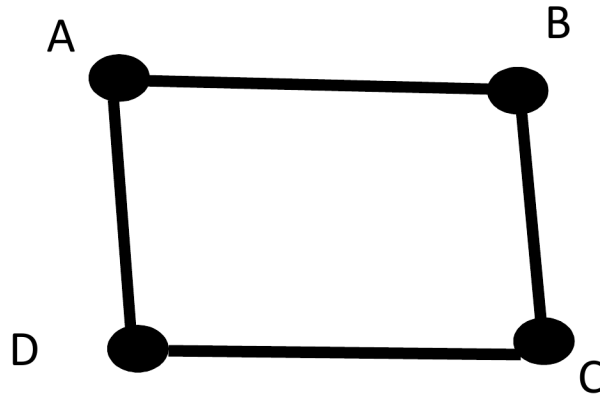


บทที่ 10

กราฟไม่มีทิศทาง Undirect G



$V = \{ A, B, C, D \}$

$E = \{ (A,B), (B,C), (C,D), (D,A) \}$

จงเขียน path จาก vertex A ไป vertex D

A, B, C, D (วิธีความยาว 3 หรือเขียนทางตรง A, D ก็ได้คะแนน)

อย่าเขียน $A \rightarrow B \rightarrow C \rightarrow D$ 0 คะแนน

Connect = ต่อเนื่อง ไม่ใช่ ต่อเชื่อม

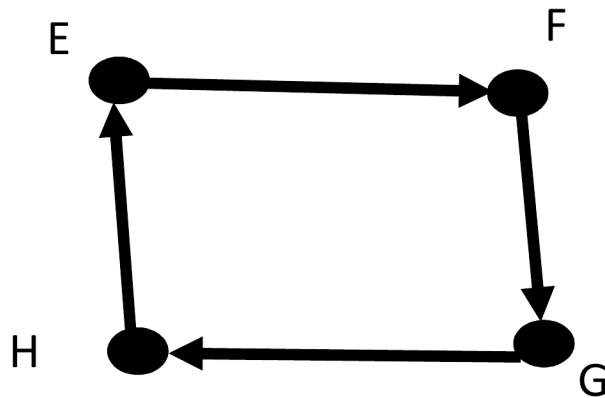
AB : วิธี A, B (ตรง L=1)	BC : B, C (ตรง L=1)	CA : C, B, A (L=2)	DA : D, A (ตรง L=1)
AC : วิธี A, B, C (L=2)	BD : B, C, D (L=2)	CB : C, B (ตรง L=1)	DB : D, A, B (L=2)
AD : วิธี A, D (ตรง L=1)	BA : B, A (ตรง L=1)	CD : C, D (ตรง L=1)	DC : D, C (ตรง L=1)

สรุปหลักการเฉลยตาราง Connect (Connected Graph):

- นิยาม Connect: ทุกคู่จุดยอดต้องมีวิถี (Path) เชื่อมถึงกันได้ แม้ไม่มี Edge ต่อตรงก็ตาม
- วิถีทางตรง (L=1): AB, AD, BC, BA, CB, CD, DA, DC (เดินผ่านเส้นเชื่อม 1 เส้น)
- วิถีต่อเนื่อง (L=2): AC (เดินผ่าน B), BD (เดินผ่าน C), CA (เดินผ่าน B), DB (เดินผ่าน A)

✓ กฎเหล็กห้องสอบ: การเขียน Path ในวิชานี้ต้องใช้ Comma คั่น เช่น A, B, C ห้ามเขียนลูกศร $A \rightarrow B$ เด็ดขาด

กราฟมีทิศทาง Direct G



$V = \{ E, F, G, H \}$

$E = \{ (E, F), (F, G), (G, H), (H, E) \}$

F adjacent E เพราะมี คู่อันดับ (E, F)

แต่ E ไม่ได้ adjacent กับ F เพราะไม่มี คู่อันดับ (F, E)

จงเขียนเส้นทาง(path) จาก จุด E ไป G

E, F, G (เดินตามลูกศร E → F → G แต่เขียนคั่นด้วย Comma เท่านั้น)

อย่าเขียน E → F → G 0 คะแนน

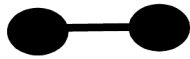
Connect

EF : E, F (L=1)	FE : F, E (เส้นเพิ่ม L=1)	GE : G, H, E (L=2)	HE : H, E (L=1)
EG : E, F, G (L=2)	FG : F, G (L=1)	GF : G, H, E, F (L=3)	HF : H, E, F (L=2)
EH : E, F, G, H (L=3)	FH : F, G, H (L=2)	GH : G, H (L=1)	HG : H, E, F, G (L=3)

🔴 สรุปหลักการ Strongly Connected ใน Directed Graph:

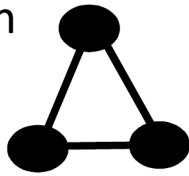
1. Directed Path: ต้องเดินตามทิศทางหัวลูกศรเท่านั้น ห้ามเดินย้อนศรเด็ดขาด
 2. กราฟนี้เป็น Strongly Connected Graph เพราะทุกจุดยอดสามารถเดินทางไป-กลับถึงกันได้ครบทุกคู่
 3. ความสำคัญของเส้นเชื่อม (F, E): หากไม่มีเส้นนี้ F, G, H จะไม่สามารถวนกลับมาถึง E ได้
 4. นิยาม Adjacent: F adjacent E เพราะมี (E, F) และ E adjacent F เพราะมี (F, E)
- ✓ ห้ามเขียนลูกศร E → F → G ในข้อสอบเด็ดขาด ต้องใช้เครื่องหมายจุลภาคคั่นเท่านั้น (E, F, G)

Complete Graph



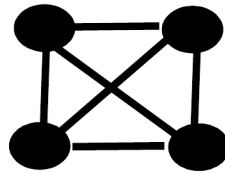
2 vertex

มี edge 1 เส้น



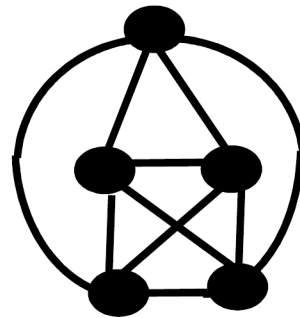
3 vertex

มี edge 3 เส้น



4 vertex

มี edge 6 เส้น



5 vertex

มี edge 10 เส้น

$$N * (N - 1) / 2$$

$$2 * (2 - 1) / 2 = 1 \text{ เส้น}$$

$$3 * (3 - 1) / 2 = 3 \text{ เส้น}$$

$$4 * (4 - 1) / 2 = 6 \text{ เส้น}$$

$$5 * (5 - 1) / 2 = 10 \text{ เส้น}$$

Complete Graph ที่มี 10 vertex ถ้ามองว่ามี edge กี่ เส้น

📌 แสดงวิธีทำ: แทนค่าสูตร $N(N - 1) / 2$ สำหรับ 10 จุดยอด

$$\text{edges} = 10 \times (10 - 1) / 2 = (10 \times 9) / 2 = 90 / 2$$

ตอบ: มี edge ทั้งหมด 45 เส้น



สรุปความรู้ Complete Graph & ข้อควรระวังในห้องสอบ:

1. นิยาม: Complete Graph (K_n) คือกราฟที่มีเส้นเชื่อมครบทุกคู่จุดยอด

2. เปรียบเทียบ **Digraph Complete Graph**: หากเป็นกราฟมีทิศทาง สูตรคือ $N(N - 1)$ (ไม่หาร 2) = 90 เส้น

3. เปรียบเทียบกับ Tree (กราฟต้นไม้): กราฟต้นไม้ N จุดยอด จะมี edge เสมอ = $N - 1$ เส้น (10 โหนดมี 9 เส้น)

4. ดีกรีของทุกโหนดใน K_n : แต่ละโหนดจะมี degree เท่ากันหมด = $N - 1$ (สำหรับ K_{10} ทุกโหนดมี $\text{deg} = 9$)

Representation of Graph

Adjacency Matrix

u\v	1	2	3	4	5	6	7
1	0	1	1	1	0	0	0
2	0	0	0	1	1	0	0
3	0	0	0	0	0	1	0
4	0	0	1	0	0	1	1
5	0	0	0	1	0	0	1
6	0	0	0	0	0	0	0
7	0	0	0	0	0	1	0

Adjacency Matrix ไม่นิยมใช้ เพราะ เปลืองเนื้อที่ในหน่วยความจำ

49 ช่อง มีช่องที่เป็น 1 อยู่ 12 ช่อง

100 ช่อง มีช่องที่เป็น 1 อยู่ = $(12/49)*100 = 24.48 \%$

มีช่องที่เป็น 0 หรือไม่ได้ใช้งาน อยู่ 75.52%



วิเคราะห์เปรียบเทียบการใช้หน่วยความจำ (Memory Efficiency Analysis):

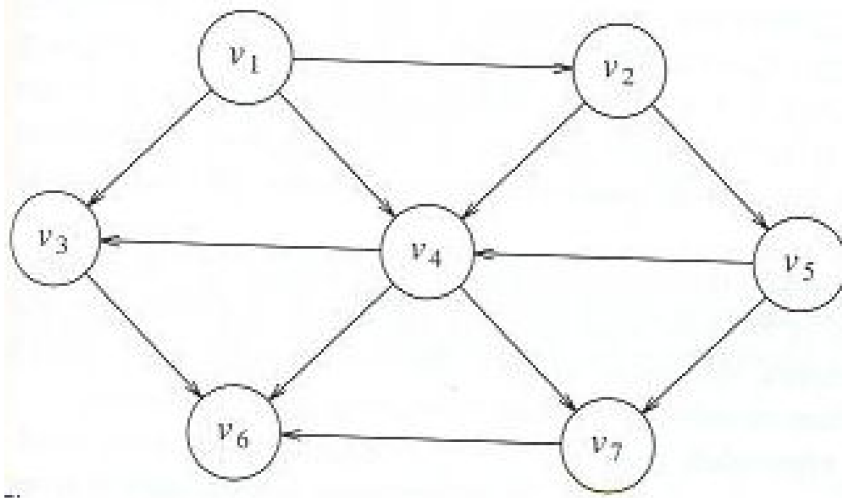
รายการคำนวณ	Adjacency Matrix (ตาราง 2 มิติ)	Adjacency List (Linked List)
พื้นที่ทางทฤษฎี	$\Theta(V ^2) = 7 \times 7 = 49$ ช่อง	$\Theta(V + E) = 7 + 12 = 19$ ช่อง
สัดส่วนการใช้จริง	24.48% (มีเส้นเชื่อมจริง 12 ช่อง)	100% (เก็บเฉพาะเส้นเชื่อมจริง)
ความสูญเปล่าใน RAM	75.52% (เสีย RAM เก็บเลข 0 ฟรี 37 ช่อง)	0% (ไม่มีช่องสูญเปล่า)
RAM ที่ใช้จริง (Bytes)	$49 \times 4B = 196$ Bytes	$7 \times 8B + 12 \times 12B = 200$ Bytes
ความเร็วค้นหาเพื่อนบ้าน	$O(V)$ ถ้า ต้องสแกนทั้งแถว 7 ช่อง	$O(\deg(v))$ เร็วมาก วิ่งตาม Pointer



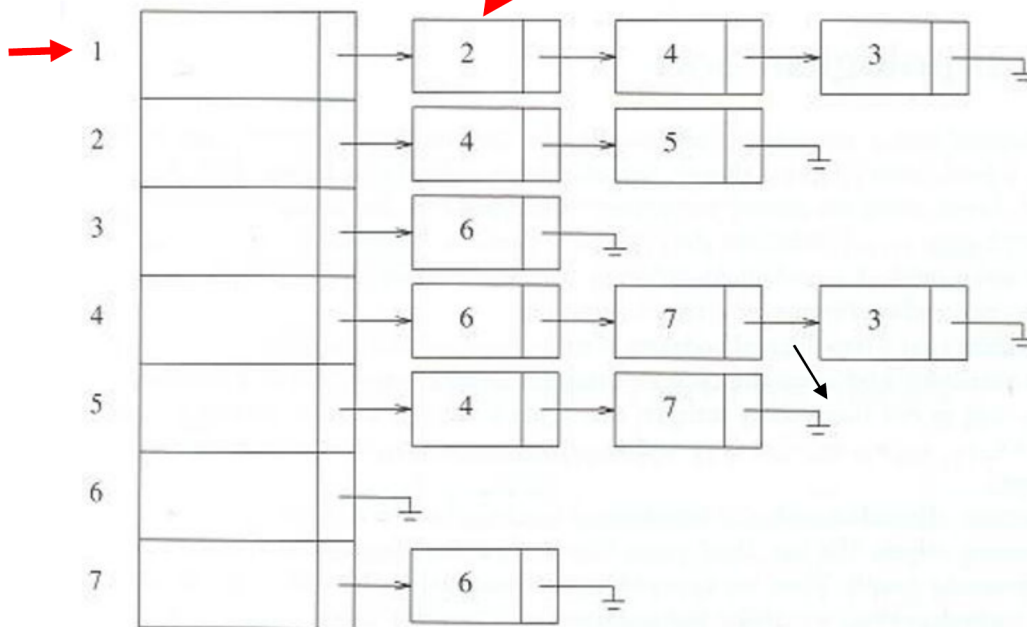
ทำไมสไลด์ถึงระบุว่า: 'Adjacency Matrix ไม่นิยมใช้ เพราะ เปลืองเนื้อที่ในหน่วยความจำ'?

- Sparse Graph ในโลกจริง: กราฟส่วนใหญ่ เช่น Social Network หรือ Web Graph เป็นแบบเบาบาง ($|E| \ll |V|^2$) หากมี 10,000 จุดยอด Matrix จะต้องจอง RAM ถึง 100,000,000 ช่อง (เสียพื้นที่เก็บเลข 0 ไปเปล่าๆ 99%!)
 - Adjacency List แก้ปัญหานี้: โดยการจองเฉพาะโหนดที่มีเส้นเชื่อมอยู่จริงเท่านั้น จึงประหยัดหน่วยความจำได้มหาศาล
 - สรุปสำหรับทำข้อสอบ: ในโจทย์นี้ Adjacency List ใช้ 19 ช่อง เทียบกับ Matrix 49 ช่อง ประหยัดกว่า 61.22%!
- ✓ ข้อดีข้อเดียวของ Matrix คือตรวจสอบ $\text{hasEdge}(u, v)$ ได้ใน $O(1)$ แต่การท่องกราฟ (Traverse) ช้ากว่า List มาก

Topological Sort



1. แปลงภาพกราฟ มาเป็น **adjacency list** ก่อน



💡 วิเคราะห์โครงสร้างหน่วยความจำ Adjacency List:

1. อาร์เรย์ Header Pointers (ขนาด 7 ช่อง, Index 1..7):

ทำหน้าที่เก็บพอยน์เตอร์ชี้ไปยังโหนดแรกของ Linked List ของแต่ละจุดยอดต้นทาง

2. โหนด Linked List แต่ละก้อนเก็บ 2 ส่วน:

[Dest (ปลายทาง) | Next* (พอยน์เตอร์ชี้ก้อนถัดไป)]

3. สัญลักษณ์กราวด์ (⊥):

แทนค่าพอยน์เตอร์ NULL บ่งชี้ว่า 'สิ้นสุดรายการเพื่อนบ้านของโหนดนี้'

4. รวมใช้พื้นที่ Memory ทั้งหมด:

$|V| + |E| = 7 + 12 = 19$ ช่อง (ประหยัดหน่วยความจำกว่า Matrix ถึง 61.22%!)

5. ลำดับโหนดใน Linked List:

ขึ้นกับการ Insert เช่น 2 -> 4 -> 3 หรือ 2 -> 3 -> 4 ถูกต้องทั้งคู่ และได้ลำดับ TopSort ที่ถูกต้อง

✓ สรุป: Adjacency List เป็นโครงสร้างที่เหมาะสมที่สุดสำหรับอัลกอริทึม Topological Sort

2. Indegree Array อะเรย์ของจำนวน edge ที่ชี้เข้าหา vertex

v	เริ่มต้น	รอบที่ 1	รอบที่ 2	รอบที่ 3	รอบที่ 4	รอบที่ 5	รอบที่ 6	q	
1	0	-	-	-	-	-	-	6	6: Enqueue(6)
2	1	0	-	-	-	-	-	5	3: Enqueue(3)
3	2	1	1	1	0	-	-	4	7: Enqueue(7)
4	3	2	1	0	-	-	-	3	4: Enqueue(4)
5	1	1	0	-	-	-	-	2	5: Enqueue(5)
6	3	3	3	3	2	1	0	1	2: Enqueue(2)
7	2	2	2	1	0	0	-	0	1: เริ่มต้น

สถานะพอยน์เตอร์เมื่อจบการทำงาน:

- Currentsize = 0 (คิวว่างเปล่าเมื่อจบ)
- Front = 0 (ดัชนีรอบครบกลับมาที่ 0)
- back = 6 (Enqueue คน 7 จุดยอด)

Indegree array `q.enqueue()` (v) // นำโหนดที่ indegree=0 เข้าคิว

`V = q.dequeue()` // ดึงโหนดออกจากคิวมาประมวลผล

for each w adjacent to v `W =` เพื่อนบ้านของ v -> ลด indegree[w] ลง 1

🔍 เจาะลึกการทำงานของ Kahn's Algorithm (Topological Sort Step-by-Step):

1. เริ่มต้น: หาจุดยอดที่มี In-degree = 0 -> ได้โหนด 1 นำเข้าคิว: `q.enqueue(1)`

2. วงวนประมวลผล Dequeue ทีละตัวจนคิวว่าง:

- รอบ 1: v=1 -> เพื่อนบ้าน w=[2,4,3] -> ลด indegree: w=2 กลายเป็น 0 จึง Enqueue(2)
- รอบ 2: v=2 -> เพื่อนบ้าน w=[4,5] -> ลด indegree: w=5 กลายเป็น 0 จึง Enqueue(5)
- รอบ 3: v=5 -> เพื่อนบ้าน w=[4,7] -> ลด indegree: w=4 กลายเป็น 0 จึง Enqueue(4)
- รอบ 4: v=4 -> เพื่อนบ้าน w=[6,7,3] -> ลด indegree: ทั้ง 7 และ 3 กลายเป็น 0 พร้อมกัน! (Enqueue 7, 3)
- รอบ 5: v=3 -> เพื่อนบ้าน w=[6] (indegree ลดเหลือ 1 ยังไม่เข้าคิว) | รอบ 6: v=7 -> w=6 กลายเป็น 0 (Enqueue 6)
- รอบ 7: v=6 -> คิวว่าง จบอัลกอริทึม!

💡 จุดสำคัญ: ในรอบที่ 4 ทั้งโหนด 3 และ 7 กลายเป็น 0 พร้อมกัน จึงสลับลำดับได้ 2 คำตอบ (ถูกต้อง 100%ทั้งคู่)
- แบบที่ 1: 1, 2, 5, 4, 3, 7, 6 | แบบที่ 2: 1, 2, 5, 4, 7, 3, 6

จงเขียน Topological Sort ที่เป็นผลลัพธ์ จากกราฟดังกล่าว

🎯 คำตอบที่ถูกต้องอย่างเป็นทางการ (Official Topological Order):

1, 2, 5, 4, 3, 7, 6 (หรือเขียน 1, 2, 5, 4, 7, 3, 6 ก็ได้คะแนนเต็มเช่นกัน ขึ้นกับลำดับการ Enqueue 3 และ 7)

⚠️ คำเตือนจุดได้ 0 คะแนน: ต้องคั่นด้วยเครื่องหมายจุลภาค (Comma) ห้ามเขียนลูกศร 1 -> 2 -> 5... เด็ดขาด!